

Planning as Autoregressive Sequence Modelling: Decoding Strategies

Dian Kruger
Shane Josias

Introduction

Planning is a central problem in artificial intelligence, concerned with finding a sequence of actions that moves an environment from an initial state to a desired goal state [6]. In this project, the planning problem is studied in a discrete 2D maze environment, where the task is to generate a valid trajectory from a given start position to a specified goal position without passing through walls.

One way to approach such a problem is to treat trajectories as data and learn a probability distribution over successful trajectories from examples. From this perspective, planning can be viewed as inference in a learned generative model [3]. If the model is trained on valid and efficient demonstration trajectories, then complete plans can be generated autoregressively by repeatedly predicting the next token in a trajectory sequence.

The decoding rule used during this generation process is important. At each step, the learned model may assign non-negligible probability to multiple possible continuations, and different decoding strategies can therefore produce trajectories with different levels of validity, efficiency, and computational cost [3, 4]. In the setting considered in this work, decoding is based on model probability rather than on a reward-maximising objective, so the planning problem is approached through imitation learning rather than offline reinforcement learning [3].

This report investigates how the choice of decoding strategy affects planning performance under this formulation. A Transformer-based trajectory model is trained through next-token prediction on a dataset of valid maze trajectories, and the resulting model is evaluated using greedy decoding and beam search with different beam widths. The aim is to determine how these decoding rules affect goal-reaching success, path efficiency, and computational cost in the maze planning setting.

Background

Generative Models

Generative models are types of machine learning systems that aim to represent a probability distribution over data, or to draw samples from that distribution [5]. Specifically, if $\mathbf{x}$ is a collection of features observed from an object or event, like words in a sentence or coordinates in a maze, and $p(\mathbf{x})$ is the true joint distribution of those features, a generative model learns an approximation $\hat{p}(\mathbf{x})$ of this distribution from training observations [5].

Once such a distribution has been learned, it can be used to generate new observations that are similar to the observations in the training data by sampling from $\hat{p}(\mathbf{x})$ [5].

In practice, assumptions are often made to simplify the learning problem, such as assuming a particular parametric form for the distribution (e.g., Gaussian), so instead of learning an arbitrary distribution, the model learns a specific family of distributions parameterised by θ [5].

If the data's true distribution $p(\mathbf{x})$ is assumed to belong to a family of distributions $\{p_\theta(\mathbf{x})\}$, the model must learn the parameter value $\hat{\theta}$ such that $p_{\hat{\theta}}(\mathbf{x})$ is a good approximation of $p(\mathbf{x})$.

One way to formalise this is through the Kullback–Leibler (KL) divergence, which measures how one probability distribution diverges from another. It is defined as

$$D_{\text{KL}}(p(\mathbf{x})\|p_\theta(\mathbf{x})) = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} \left[\log \frac{p(\mathbf{x})}{p_\theta(\mathbf{x})} \right] \quad [5]. \quad (1)$$

In this setting, learning can be viewed as choosing θ to minimise $D_{\text{KL}}(p(\mathbf{x})\|p_\theta(\mathbf{x}))$. Expanding this expression gives

$$D_{\text{KL}}(p(\mathbf{x})\|p_\theta(\mathbf{x})) = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [\log p(\mathbf{x})] - \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [\log p_\theta(\mathbf{x})]. \quad (2)$$

The first term does not depend on θ , so minimising the KL divergence is equivalent to maximising $\mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [\log p_\theta(\mathbf{x})]$, that is, the expected log-likelihood of the data under the model [5].

The process of finding $\hat{\theta}$ that maximises the likelihood of the training data under $p_\theta(\mathbf{x})$ is known as maximum likelihood estimation (MLE), and is a common approach to training generative models [5].

The Planning Problem

Broadly, automated planning is the field of research addressing the problem of finding an ordered or partially ordered series of actions that changes an environment from its initial state to a desired goal state [6].

In this work, the planning problem is to generate a valid trajectory from a given starting position to a specified goal position in a 2D Maze environment. In this environment, a trajectory consists of a sequence of maze positions from the start location to the goal location, together with the actions connecting consecutive positions, such that the path does not pass through walls. Assuming a discrete grid-based representation, maze coordinates are integers identifying grid cells. A trajectory can therefore be represented as a sequence of coordinates in which each consecutive pair corresponds to adjacent non-wall cells, with adjacency restricted to horizontal and vertical moves.

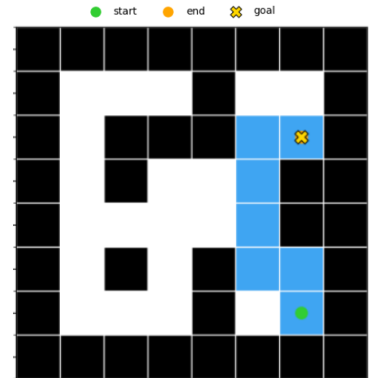


Figure 1: Example of a valid trajectory in a discrete 2D maze environment

Approaches to Planning

Reinforcement learning (RL) is a machine learning paradigm in which a system learns to choose appropriate sequences of actions by interacting with an environment in order to maximise cumulative reward, making it a natural fit for planning problems, since the reward can be defined in terms of reaching a goal state efficiently [2].

Offline reinforcement learning is a variant of RL in which the policy that determines the agent’s actions is learned from a fixed dataset of previously collected trajectories, rather than through repeated interaction with the environment [3].

Imitation learning is a related paradigm in which a model learns to reproduce the behavior demonstrated in a dataset. Similarly to offline RL, imitation learning can be applied to a dataset of trajectories, but the objective is to generate trajectories that are similar to those in the dataset, as opposed to maximising cumulative reward as in offline RL [3].

Similarly to RL, imitation learning can solve a planning problem, but only if the dataset contains trajectories that are valid and efficient for the planning task at hand [3].

Generative Modelling for Planning

If a dataset of valid trajectories is available, a generative model can be trained to learn the distribution over these trajectories. Once trained, the model can be used to generate new trajectories by sampling from the learned distribution. This approach treats planning as a problem of inference in a probabilistic model [3]. So if we define a trajectory τ with T steps as:

$$\tau = (s_1, a_1, s_2, a_2, \dots, s_T, a_T)$$



where s_t is the state (coordinates) at time step t and a_t is the action (moving up, down, left or right) taken at time step t , the trained generative model defines an approximation $p_{\hat{\theta}}(\boldsymbol{\tau})$ to the distribution over valid trajectories. Instead of directly modelling $p_{\hat{\theta}}(\boldsymbol{\tau})$, an autoregressive model factorises this distribution into a product of conditional probabilities [5]:

$$p_{\hat{\theta}}(\boldsymbol{\tau}) = \prod_{t=1}^T p_{\hat{\theta}}(z_t \mid z_{<t})$$

where z_t denotes the t^{th} token in the trajectory sequence, which may correspond to either a state or an action, and the trajectory prefix, $z_{<t}$, denotes the sequence of preceding tokens up to z_{t-1} . This simplifies the supervised learning problem to estimating $p_{\theta}(z_t \mid z_{<t})$, so given a prefix of trajectory tokens, predict the probability of the next token in the sequence.

A complete trajectory is then generated autoregressively by repeatedly predicting the next token until a full plan is produced, with the decoding strategy determining how each next-token choice is made. The choice of decoding strategy is important because the model’s predicted distribution may assign non-negligible probability mass to multiple valid continuations at each step, and the continuation selected during decoding can significantly affect both the quality of the final trajectory and the computational cost of generation [3]. Since the learned distribution is only an approximation to the true trajectory distribution, it may also assign non-negligible probability to invalid continuations. As a result, the decoding strategy can influence whether a valid trajectory is generated at all. Even when multiple valid trajectories are possible, some may be more desirable than others, for example because they require fewer steps, and the decoding strategy can therefore affect whether a more optimal trajectory is produced [3, 4].

Decoding Methods

Direct sampling from $p_{\hat{\theta}}(\boldsymbol{\tau})$ is challenging, since the number of possible trajectories grows exponentially with trajectory length, so explicitly calculating and storing the probabilities for all complete trajectories quickly becomes computationally infeasible [5]. This creates a need for decoding algorithms to approximately sample from, or search within $p_{\hat{\theta}}(\boldsymbol{\tau})$.

Greedy Decoding

Greedy decoding is the simplest method for generating sequences from $p_{\hat{\theta}}(\boldsymbol{\tau})$ [4]. At each step t during the autoregressive generation of a trajectory, the algorithm selects the next token $\hat{z}_t$ as follows:

$$\hat{z}_t = \arg \max_{z \in V} p_{\hat{\theta}}(z \mid z_{<t})$$

where V is the set of all possible tokens, and $z_{<t}$ denotes the values obtained in the previous steps of greedy decoding. Greedy decoding thus always selects the locally optimal choice for the next token in the sequence, meaning it chooses the token with the highest conditional probability given the current prefix [4]. Greedy decoding is therefore deterministic and computationally efficient, but does not necessarily find the globally most

probable trajectory, which is the complete trajectory with the highest probability under $p_{\hat{\theta}}(\boldsymbol{\tau})$ [4].

Beam Search

Beam search extends greedy decoding by retaining multiple candidate sequences at each step rather than only the single most likely continuation [4]. The number of retained candidates is fixed in advance and is known as the beam width B . At each stage, all valid one-step continuations of the current candidates are scored autoregressively under the model, and the top B highest-scoring sequences are kept. This process is repeated until termination, after which the highest-scoring complete trajectory among the retained candidates is returned.

The beam search algorithm in the context of trajectory planning can be described as follows:

Beam Search [3]

- 1: **Require** maximum sequence length T , beam width B , starting position s_{start} , and goal position s_{goal}
 - 2: **Initialize** $\mathcal{T}_0 = \{(s_{\text{start}})\}$
 - 3: **for** $t = 0, \dots, T - 1$ **do**
 - 4: For each partial trajectory $\boldsymbol{\tau} \in \mathcal{T}_t$, let $\mathcal{V}(\boldsymbol{\tau})$ denote the set of valid next tokens available from the final state of $\boldsymbol{\tau}$
 - 5: $C_{t+1} \leftarrow \{\text{append}(\boldsymbol{\tau}, z) \mid \boldsymbol{\tau} \in \mathcal{T}_t \text{ and } z \in \mathcal{V}(\boldsymbol{\tau})\}$
 // candidate single-token extensions
 - 6: $\mathcal{T}_{t+1} \leftarrow$ the set of the B highest-scoring sequences in C_{t+1} under $\log p_{\hat{\theta}}(\boldsymbol{\tau})$
 - 7: **if** any $\boldsymbol{\tau} \in \mathcal{T}_{t+1}$ ends at s_{goal} **then stop**
 - 8: **end for**
 - 9: **Return** $\arg \max_{\boldsymbol{\tau} \in \mathcal{T}_{t+1}} \log p_{\hat{\theta}}(\boldsymbol{\tau})$
-

Figure 2: Pseudocode for beam search in the trajectory planning setting.

The score $\log p_{\hat{\theta}}(\boldsymbol{\tau})$ can be accumulated autoregressively for each candidate trajectory. Since the beam width places an upper bound on the number of candidate trajectories in C_t and retained trajectories in $\mathcal{T}_t$ at each step t , maintaining these scores for all candidates remains computationally feasible when the beam width is sufficiently small [4].

Larger beam widths allow the algorithm to consider a greater number of candidate trajectories. As a result, the trajectory returned by beam search will typically be at least as highly scored under the model as the trajectory returned with a smaller beam width, with a beam width of one corresponding to greedy decoding. This broader search, however, requires the model scores of more candidate trajectories to be computed and maintained, so increasing the beam width also increases the computational cost of decoding [4].

Both greedy decoding and beam search, as defined above, aim to find trajectories with high probability under the learned model, and therefore correspond naturally to an imitation learning setting. If, however, the decoding objective is defined in terms of predicted reward or return rather than model probability alone, then the same decoding procedures can instead be used in an offline reinforcement learning setting.

Transformer Architecture



The Transformer **architecture** is a type of neural network that has become the standard for sequence modelling tasks, notably in natural language processing [4]. Even though the original Transformer was designed for language, the fact that it models sequences of tokens makes it applicable to our trajectory modelling problem, where the tokens could be states and/or actions in a trajectory [3].

The key components of the Transformer architecture are now briefly introduced. For intuition, the discussion refers primarily to natural language processing examples, where the role of these components is often easier to interpret than in trajectory modelling.

1. Input Representation:

Before raw data enters the Transformer, it must be converted into a high dimensional vector **representation**.

- **Tokenisation**: The first step involves tokenisation, where the input sequence is broken down into discrete tokens. In an NLP scenario, this could involve splitting text into words, subwords or characters [4]. For the purposes of this discussion, it is assumed that the input has been tokenised at the word level.
- **Embeddings (E)**: Each token ID is used to index into an embedding matrix E , which maps the discrete ID to a dense vector of dimensionality d . This matrix has shape $|V| \times d$, where $|V|$ is the vocabulary size, i.e. the total number of distinct tokens that the model can represent [4]. When tokens correspond to words, the embedding can be interpreted as capturing semantic and syntactic properties of those words. Intuitively, words with similar meanings, such as “big” and “large,” may be represented by vectors that are close to one another in the embedding space [4].
- **Positional Encoding**: Since the Transformer architecture processes sequences in parallel, it does not inherently encode the order of tokens, so a positional encoding is added to the token embeddings to ensure that the position of each token in the sequence is represented. This is done by adding a positional encoding vector to each token embedding [4].
- **Input Formation**: For a sequence of N tokens, the individual d -dimensional vector embeddings are placed into a single matrix X of size $N \times d$. This matrix serves as the input for the model [4].

2. Transformer Blocks:

Each block **consists** of a multi-head self-attention mechanism followed by a feedforward neural network, with each sublayer followed by a layer normalisation and a residual connection [4].

A useful mental model is that each time the input passes through a sub-layer, the model updates the token representations so that they better capture the information relevant to the task at hand. For example, in an NLP context, the model may refine the representation of a word so that it better reflects its meaning in the context of the sentence [4].



- **Layer Normalisation:**

For training stability, layer normalisation is applied to ensure the values in the sublayers remain within a reasonable range.

For a vector x , it is calculated as:

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sigma} + \beta$$

where μ is the mean, σ the standard deviation of the vector's elements, and γ and β are learned parameters that scale and shift the normalized output. [4].

- **Multi-Head Self-Attention:**

This is the core mechanism that allows the model incorporate information from all tokens in the sequence when processing each token [4]. Under the interpretation that each sub-layer progressively refines word representations, the self-attention mechanism allows the representation of each word to be updated in light of the representations of all other words in the sequence. For example, if the word “fly” is preceded by the word “the”, the model may adjust its representation in a way that reflects the nominal sense of “fly” (the insect) rather than the verbal sense. Each input vector x_i is mapped into three learned representations: a Query (Q), a Key (K), and a Value (V). For an input matrix X , these are obtained by learned linear projections:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

where W_Q , W_K , and W_V are learned weight matrices [4].

The attention for a single head is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

The scaling factor $\sqrt{d_k}$ prevents the dot products from growing too large and causing numerical instability. In causal (left-to-right) models, such as predicting the next word in a sentence, or the next position in a trajectory, a mask is applied to ensure that tokens can only attend to the past, preventing the model from “cheating” by looking at future tokens during training [4].

- **Feedforward Neural Network:**

After the attention layer incorporates information across tokens, the Feedforward Network (FFN) processes each token position independently and in parallel [4]. It is usually a two-layer network with a non-linear activation such as ReLU:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

This layer allows the model to perform more complex transformations of the contextualised information gathered by the attention heads [4].

Returning to the intuitive picture introduced earlier, the attention mechanism can be viewed as first updating each word representation using information from the other words in the sequence. The FFN then further refines each of these updated representations independently at each position, using transformations learned from many training sequences.

3. Prediction Head:

After the data passes through the final transformer block, the final representations are passed to the next-token prediction head, which converts them into a probability distribution over possible next tokens in the sequence.

- **Output Projection:** Let $\mathbf{h}_t$ denote the final hidden representation produced after the model has processed the prefix up to position t . This d -dimensional vector summarises the information in the observed prefix and is then projected back into the vocabulary space, often by multiplying it by the transpose of the embedding matrix E^T [4].
- **Softmax:** The resulting logits are passed through a softmax function to produce a probability distribution over the next token:

$$\mathbf{p}_{t+1} = \text{softmax}(\mathbf{h}_t E^T)$$

where $\mathbf{p}_{t+1}$ is the probability distribution over possible values of the next token z_{t+1} given the prefix $z_{\leq t}$ [4].

In general, this means that the model assigns a probability to each possible next token given the preceding trajectory prefix. In an NLP setting, where tokens correspond to words, this can be interpreted as the model estimating which word is most likely to come next in the sentence. The next token is then selected according to a decoding strategy such as greedy decoding, sampling, or beam search.

Methodology

Problem Formulation

The objective of this work is to model maze planning as autoregressive inference in a learned conditional trajectory model trained through next-token prediction. Given a maze

layout, a start state, a goal state, and a trajectory prefix, the model predicts the next token in the trajectory sequence. At inference time, decoding algorithms are then used to construct complete trajectories by selecting continuations with high probability under the learned model. Since decoding is based on model probability rather than a reward-maximising objective, this amounts to using imitation learning to solve the planning problem.

A trajectory $\tau = (z_1, z_2, \dots, z_{2T})$ consists of alternating state and action tokens, $(s_1, a_1, s_2, a_2, \dots, s_T, a_T)$, where the states represent coordinates in the maze and the actions correspond to the moves (up, down, left, or right) between consecutive states. In this environment, the state immediately following an action is fully determined by that action and the preceding state. Supervision is therefore applied only at action-token positions, so the learning problem is more precisely to predict the next action conditioned on the maze layout, the start state, the goal state, and the previously observed trajectory prefix.

Let m denote a maze layout identifier, let s_{start} and s_{goal} denote the start and goal states. The model therefore learns conditional predictions of the form

$$p_{\hat{\theta}}(z_{t+1} \mid m, s_{\text{start}}, s_{\text{goal}}, z_{\leq t}),$$

or, since supervision is applied only at action-token positions, equivalently

$$p_{\hat{\theta}}(a_t \mid m, s_{\text{start}}, s_{\text{goal}}, s_1, a_1, \dots, s_t),$$

These conditional predictions define an autoregressive factorisation of the conditional distribution over action sequences:

$$p_{\hat{\theta}}(a_1, a_2, \dots, a_T \mid m, s_{\text{start}}, s_{\text{goal}}) = \prod_{t=1}^T p_{\hat{\theta}}(a_t \mid m, s_{\text{start}}, s_{\text{goal}}, s_1, a_1, \dots, s_t).$$

Ideally, generating a complete plan would require sampling from the full joint conditional distribution over action sequences above; in practice, the decoding algorithms instead operate on its autoregressive conditional factors.

A Transformer was selected as the generative model of choice because of its strong performance on sequence modelling tasks, the wide range of Python libraries available for working with it, and its computational efficiency when the architecture is kept small. The research question, however, is not whether a Transformer can represent such a distribution in principle, but how different decoding strategies affect the quality of the generated plans once the model has been trained. In particular, the methodology compares greedy decoding and beam search in terms of trajectory validity, goal-reaching success, path efficiency, and computational cost.

Dataset Construction

Since the goal is to train a Transformer to predict the next action given a trajectory prefix, the training data must consist of complete trajectories τ . A trajectory token sequence

provides multiple supervised training examples, since for each token position k in a given trajectory, the prefix

$$(z_1, z_2, \dots, z_{k-1})$$

can be used to predict the next token z_k .

In addition to the trajectory tokens themselves, the model input must encode which maze the trajectory belongs to, as well as the associated start and goal positions.

In this work, eight different maze layouts are considered, corresponding to the layouts used in the D4RL PointMaze / Maze2D environment. The exact layout definitions can be found in the D4RL repository at https://github.com/Farama-Foundation/D4RL/blob/master/d4rl/pointmaze/maze_model.py.

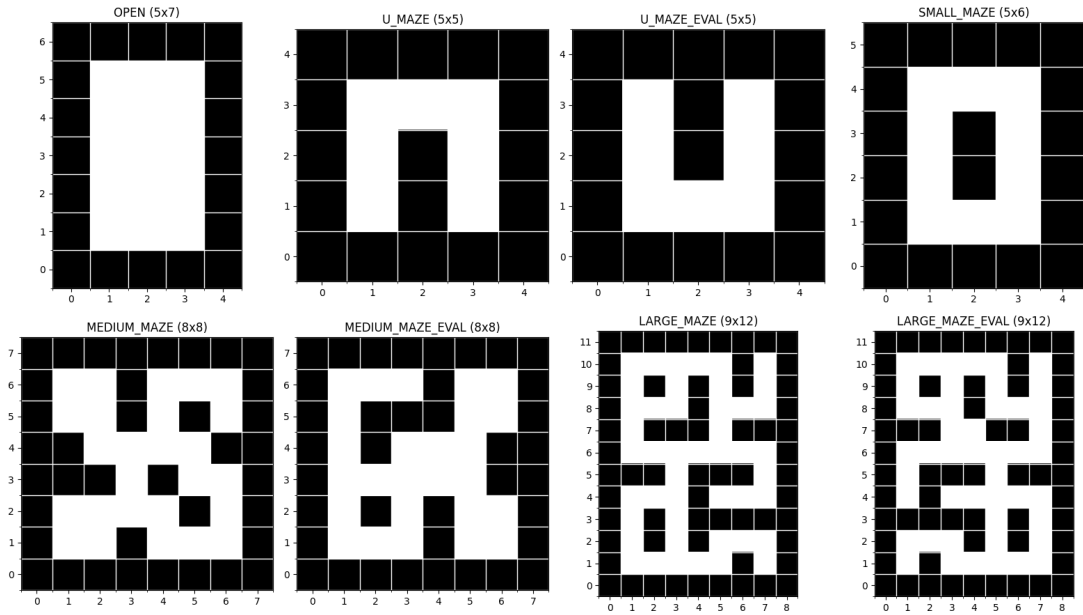


Figure 3: Maze layouts used from the D4RL PointMaze / Maze2D environment.

In `maze_model.py`, each maze is defined as a matrix, and the row and column indices of this matrix determine the corresponding grid cells in the discrete maze representations used throughout this work.

To represent the states, actions, and maze layouts as Transformer inputs, each is encoded as a discrete integer token. Distinct token ranges are reserved for different token classes by defining token offsets, so that maze tokens, state tokens, and action tokens occupy non-overlapping regions of the overall vocabulary.

States are encoded by assigning each grid location a unique integer index. This is done using the mapping

$$\text{state_index} = \text{row} \times (\text{max columns}) + \text{column},$$

where *row* and *column* are the integer indices of the grid location, and *max columns* is the maximum number of columns across all maze layouts. This produces a unique integer identifier for each location in the global grid. A state token is then obtained by adding a

fixed state-token offset to this index. Although this indexing scheme assigns identifiers to all grid locations in the global grid, including wall locations, only valid free-cell positions appear in the generated trajectory sequences.

Actions are also encoded as integer tokens. Since the trajectories are restricted to four-connected grid movement, the action vocabulary consists of the four moves up, down, left, and right, with each assigned its own unique integer token identifier.

In the training data, the detailed wall structure of a maze is not included explicitly in the token sequence. Instead, each trajectory is associated with a single integer maze token identifying which of the eight layouts it belongs to. The model must infer the relevant layout constraints from this maze identifier together with the observed trajectory tokens. This design choice gives a simple and efficient encoding scheme, but it also means that the model is tied to the set of maze layouts seen during training and cannot directly generalise to previously unseen layouts without additional representation of maze geometry.

The tokenised input sequences are stored in an input matrix X of dimensions $N \times L$, where N is the number of training trajectories and L is the maximum token-sequence length observed in the training dataset.

The final form of each row $\mathbf{x}$ of the input matrix X is

$$\mathbf{x} = (m, s_{\text{start}}, s_{\text{goal}}, s_1, a_1, s_2, a_2, s_3, \dots, a_T, s_{T+1}, 0, \dots, 0).$$

where s_{start} and s_{goal} are represented using the same coordinate encoding as the states appearing in the trajectory. The sequence $(s_1, a_1, s_2, a_2, \dots, s_T, a_T)$ represents the alternating state and action tokens that make up the trajectory. Any unused entries at the end of the row are filled with padding tokens (0) so that all rows of X have the same length.

The trajectories used in the training data were generated through the following process. For the maze layouts used in this work, the number of free (non-wall) grid squares in a maze ranges from 7 to 47. If a maze has n free squares, then there are $n(n-1)$ ordered start-goal combinations, since the start and goal positions must be distinct.

For each such start-goal pair, an optimal trajectory is generated using breadth-first search (BFS). Here, optimal means that the resulting path minimises the number of adjacent grid moves required to connect the start and goal positions. Since movement is restricted to adjacent non-wall squares, BFS is sufficient to recover a shortest valid path in this discrete setting. Because the number of start-goal combinations for each maze is relatively small, generating shortest paths for all pairs is computationally inexpensive.

The BFS output provides a sequence of states, that is, a sequence of grid positions forming a shortest path from the start state to the goal state. The corresponding sequence of actions can then be computed directly from consecutive state differences. In this way, each start-goal pair yields a complete trajectory consisting of alternating state and action information, and these trajectories form the basis of the Transformer’s training data.

To compare decoding strategies on unseen queries, the transformer-ready dataset is partitioned using 5-fold cross-validation. The dataset contains exactly one trajectory for each ordered start-goal pair that is connected by a valid shortest path in a given maze. Within each maze separately, the trajectories are **sorted** deterministically by path length and then

lexicographically by start and goal coordinates. Folds are then assigned in round-robin order over this sorted list. Because each start-goal pair appears only once in the dataset, no exact query appears in both the training and evaluation portions of a given fold. In the decoding benchmark, evaluation is reported on the five maze families **OPEN**, **U_MAZE**, **SMALL_MAZE**, **MEDIUM_MAZE**, and **LARGE_MAZE**. The aim of the experiment is therefore to compare how different decoding rules behave on unseen start-goal planning queries under a fixed trained model class.

Transformer Training

In addition to the input matrix X , the tokenised input sequences are stored in an input matrix X , the token-type information is stored in a token-type matrix T , the target outputs are stored in a label matrix Y , and the padding information is stored in a mask matrix M . T , Y and M are all of the same shape as X . Each entry in the token-type matrix indicates whether the corresponding token is a maze token, start token, goal token, state token, action token, or padding token. In the label matrix, only action-token positions contain active targets, while all other positions are ignored during loss computation. The mask matrix indicates which positions in the input sequences are padding tokens.

A small causal Transformer was trained on the tokenised trajectory datasets. After the model receives the training data, token embeddings and token-type embeddings are summed, positional encodings are added, and the resulting sequence is processed by a stack of causal self-attention layers so that each token can depend only on the current and preceding tokens in the sequence.

The model was kept small for faster training. The architecture used a hidden dimension of 64, 4 attention heads, 2 Transformer layers, a feedforward dimension of 128, and dropout of 0.1. Training was performed using the AdamW optimiser with learning rate 3×10^{-4} and weight decay 10^{-4} . For each training fold, the model was trained for 20 epochs on the training partition of the fold, with a batch size of 128. After every epoch, the model was evaluated on both the training and validation partitions of the fold. The checkpoint with the best validation loss was saved and used for decoding evaluation.

The training objective was cross-entropy loss applied to next-token prediction, with supervision active only at action-token positions. Cross-entropy was chosen because the target at each supervised position is a discrete action token drawn from a finite vocabulary, so the learning problem is naturally one of categorical prediction. Non-action positions in the label matrix Y were ignored during loss computation.

Decoding

The decoding strategies compared in this work are greedy decoding and beam search with beam widths 2 and 3. The trained Transformer predicts a distribution over the next action token conditioned on the maze token, start token, goal token, and the trajectory prefix generated so far. Decoding then determines how these conditional predictions are converted into a complete trajectory.

For a given evaluation query, decoding begins from the token prefix consisting of the maze identifier, the start token, the goal token, and the initial state token. The model is then queried autoregressively for the next action token. In the implementation used here, the model’s logits are not accepted blindly over the full action vocabulary. Instead, at each step, the candidate actions are restricted to the actions that lead to valid neighbouring free cells from the current state. In other words, candidate moves that would pass through walls or leave the maze are excluded before the decoding rule is applied. Once an action has been selected, the corresponding next state is deterministically computed from the current state and appended to the trajectory prefix, and the procedure repeats until the goal is reached or a step limit is exceeded.

The step limit refers to the maximum number of decoding steps, equivalently the maximum number of action predictions, that a decoding algorithm is allowed to generate for a query before termination is forced if the goal has not yet been reached. In the implementation used here, this limit is fixed at 19 steps for all queries. This value was chosen because the Transformer was trained with a maximum input sequence length of 42 tokens, while the initial decoding prefix already occupies four tokens: the maze token, the start state token, the goal state token, and the initial state token. Each subsequent decoding step appends one action token and one next-state token, so the largest number of decoding steps that can be represented without exceeding the model’s maximum sequence length is

$$\left\lfloor \frac{42 - 4}{2} \right\rfloor = 19.$$

The fixed step limit of 19 therefore ensures that decoding remains consistent with the Transformer’s input size while allowing the longest rollout permitted by the model.



Since the Transformer only provides an estimated conditional distribution over next actions, a decoding strategy may fail to reach the goal within the allowed horizon even when a valid path exists. For this reason, the first evaluation metric is **Completion Rate**, defined as the proportion of evaluation queries for which the decoded trajectory reaches the goal within the allowed number of steps.

The second metric is **Average Steps to Goal**. In the implementation, this is computed as the mean number of executed actions, equivalently the number of state transitions, over successful queries only. This metric measures the efficiency of the trajectories produced by a decoding strategy, conditional on success.

The remaining two metrics quantify computational cost. For each decoded trajectory, the total wall-clock time required to perform decoding is recorded. This yields **Average Compute Seconds**, defined as the mean total decoding time per evaluation query. In addition, the total decoding time for a query is divided by the number of decoded actions taken for that query, yielding a per-query seconds per step quantity. Averaging this value over evaluation queries gives **Average Seconds per Step**. Together, these four metrics allow the comparison to capture both planning quality and computational efficiency.

Results

The following table reports the overall decoding performance across all eight maze layouts, aggregated over the full cross-validation evaluation.

Table 1: Dataset-wide decoding results aggregated across all eight maze layouts.

Strategy	Completion Rate	Avg. Steps to Goal	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	0.6916	7.7205	0.01766	0.00155
Beam 2	0.9100	7.0771	0.01401	0.00167
Beam 3	0.9754	6.9058	0.01298	0.00173

Across all five cross-validation folds, the evaluation set contained 5976 trajectory records in total, distributed as follows: OPEN (240), U_MAZE (90), U_MAZE_EVAL (90), SMALL_MAZE (24), MEDIUM_MAZE (650), MEDIUM_MAZE_EVAL (650), LARGE_MAZE (2070), and LARGE_MAZE_EVAL (2162).

Larger mazes consequently contribute more records to the overall evaluation, so the aggregated results in Table 1 are dominated by the performance on the larger maze layouts. Appendix A reports the performance of the decoding strategies for each individual maze layout.

Discussion and Conclusion

It is observed in Table 1 that beam search with width $B = 2$ and $B = 3$ outperforms greedy decoding in terms of both completion rate and average steps to goal, with the larger beam width yielding the best overall performance. This is consistent with the expectation that a broader search over candidate trajectories yields trajectories with higher probability under the learned model, and since the model was trained on optimal trajectories, these higher-probability trajectories are more likely to correspond to valid and efficient paths to the goal.

The trade-off typically associated with beam search, relative to greedy decoding, and with increasing beam width, is that the broader search requires more computation. The average time per step shown in Table 1 is indeed higher for beam search than for greedy decoding, and in the aggregate it also increases with beam width. However, in this implementation the average total compute time per query is lower for beam search than for greedy decoding, and is also lower for $B = 3$ than for $B = 2$. This can be explained by the fact that broader searches reach the goal in fewer steps on average, so the reduction in the number of decoded steps outweighs the increase in time per step, yielding a lower total compute time.

The performance of the decoding strategies did vary across maze layouts, as reported in Appendix A. However, the relative ordering in completion rate and average steps to goal was consistent across all maze layouts: beam search performed at least as well as greedy decoding, and $B = 3$ performed at least as well as $B = 2$. For average time per step, the same overall pattern also held in almost all cases. The only exception was the U_MAZE

layout, where beam search with width $B = 3$ had a slightly lower average time per step than beam search with width $B = 2$.

In some mazes, the average total compute time did not decrease monotonically with beam width. This occurred in layouts where there were typically fewer valid path options from a given state, so the average steps to goal of the broader searches were close to, or equal to, those of the narrower searches. In those cases, the additional per-step cost of the broader search was not always offset by a shorter decoded trajectory.

In conclusion, the results demonstrate that beam search is a more effective decoding strategy than greedy decoding for the maze planning problem under the learned trajectory model, both in terms of goal-reaching success and path efficiency. The strongest results were obtained with beam width $B = 3$, which achieved the highest completion rate and the lowest average steps to goal across the full evaluation.

Future Work

The current work has focused on comparing decoding strategies, specifically greedy decoding and beam search, when solving the maze planning problem through imitation learning under a learned trajectory model. One underlying assumption is that the model has been trained on optimal trajectories, so that higher-probability trajectories under the model are more likely to correspond to valid and efficient paths to the goal. In future work, it would be interesting to explore the performance of different decoding strategies when the model is trained on sub-optimal trajectories. The expectation is that, under the current imitation learning setting, the performance of the decoding strategies will degrade when the model is trained on sub-optimal trajectories, since higher-probability trajectories under the model will no longer be guaranteed to be valid and efficient paths to the goal.

A natural extension would then be to study decoding strategies in an offline reinforcement learning setting, where the decoding objective is defined in terms of predicted reward or return rather than model probability alone. This is expected to improve the performance of the decoding strategies when the model is trained on sub-optimal trajectories.

More decoding strategies can also be added to the current work, in addition to the extensions mentioned above. Strategies such as stochastic decoding or nucleus sampling could be explored and compared with greedy decoding and beam search.

References

- [1] P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [2] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.

- [3] Michael Janner, Qiyang Li, and Sergey Levine. Offline Reinforcement Learning as One Big Sequence Modeling Problem. *Advances in Neural Information Processing Systems*, 2021.
- [4] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition. Online manuscript released January 6, 2026. https://web.stanford.edu/~jurafsky/slp3/ed3book_jan26.pdf
- [5] Ian Goodfellow, Yoshua Bengio and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [6] Diaeddin Alarnaouti, George Baryannis, Mauro Vallati. *Reformulation techniques for automated planning: a systematic review*. *The Knowledge Engineering Review*, 38, 2023.

A Per-Maze Decoding Results

This appendix reports decoding performance for each maze layout.

Table 2: Decoding results for the OPEN maze.

Strategy	Completion Rate	Avg. Steps to Goal	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	0.829167	5.783920	0.013712	0.001710
Beam 2	0.950000	3.982456	0.009213	0.001801
Beam 3	0.995833	3.372385	0.006854	0.001875

Table 3: Decoding results for the U_MAZE maze.

Strategy	Completion Rate	Avg. Steps to Goal	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	0.733333	4.015152	0.013162	0.001640
Beam 2	0.977778	3.715909	0.006804	0.001675
Beam 3	1.000000	3.688889	0.006152	0.001654

Table 4: Decoding results for the U_MAZE_EVAL maze.

Strategy	Completion Rate	Avg. Steps to Goal	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	0.866667	4.371795	0.009952	0.001534
Beam 2	1.000000	3.666667	0.006119	0.001554
Beam 3	1.000000	3.666667	0.006166	0.001596

Table 5: Decoding results for the SMALL_MAZE maze.

Strategy	Completion Rate	Avg. Steps to Goal	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	1.000000	2.250000	0.003233	0.001482
Beam 2	1.000000	1.666667	0.002592	0.001558
Beam 3	1.000000	1.666667	0.002690	0.001572

Table 6: Decoding results for the MEDIUM_MAZE maze.

Strategy	Completion Rate	Avg. Steps to Goal	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	0.749231	6.478439	0.015635	0.001580
Beam 2	0.933846	5.367381	0.010951	0.001695
Beam 3	0.989231	5.202177	0.009521	0.001723

Table 7: Decoding results for the MEDIUM_MAZE_EVAL maze.

Strategy	Completion Rate	Avg. Steps to Goal	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	0.629231	6.051345	0.016919	0.001514
Beam 2	0.901538	5.373720	0.011219	0.001622
Beam 3	0.967692	5.166932	0.009714	0.001679

Table 8: Decoding results for the LARGE_MAZE maze.

Strategy	Completion Rate	Avg. Steps to Goal	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	0.665700	8.601597	0.018935	0.001541
Beam 2	0.909179	8.172157	0.015630	0.001667
Beam 3	0.985024	7.997057	0.014805	0.001750

Table 9: Decoding results for the LARGE_MAZE_EVAL maze.

Strategy	Completion Rate	Avg. Steps to Goal	Avg. Compute Time (s)	Avg. Time per Step (s)
Greedy	0.690102	8.455094	0.018373	0.001532
Beam 2	0.894080	7.808070	0.015510	0.001671
Beam 3	0.959759	7.636145	0.014605	0.001736